

# High-Performance BLAS Library Development

## Level 1, 2, 3 Kernels and 3D Task-Based Parallelism

Antigravity AI & Student Team

April 24, 2026

## Objectives:

- Full BLAS suite implementation.
- Rival OpenBLAS performance.
- Use AI-driven iterative tuning.

## Core Technologies:

- **SIMD:** AVX2 + FMA (256-bit).
- **Parallelism:** OpenMP Tasks.
- **Optimization:** Tile-based loops, NT stores, software prefetching.

# BLAS 1: Vector Operations

## What are they?

- **sscal**:  $x = \alpha x$  (Scalar product).
- **saxpy**:  $y = \alpha x + y$  (Constant times vector plus vector).
- **sdot**:  $s = \sum x_i y_i$  (Dot product).

## Intermediate Optimization Results (Strided Access):

- **Problem**: Stride  $\neq 1$  prevents direct AVX2 vectorization.
- **Week 4 Baseline**: 3.2 GB/s (Naïve scalar loop).
- **Week 5 Improvement**: 3.9 GB/s (**Unroll-by-4 Scalar Path**).
- **Gain**: **+21%** speedup by improving instruction pipeline residency.

# BLAS 2: Matrix-Vector Operations

## Definitions

- **sgemv**:  $y = \alpha Ax + \beta y$  (General matrix-vector product).
- **sger**:  $A = \alpha xy^T + A$  (Rank-1 update; matrix addition from vector outer product).
- **ssyr / ssyr2**: Symmetric rank-1/2 updates.  $A = \alpha xy^T + \alpha yx^T + A$ .

## Key Optimization:

- **Task-Based Parallelism**: Switched from static for-loops to `omp taskloop`. Provides better load balancing for tall/skinny matrices.
- **Branch Elimination**: Decoupled triangular boundary checks from inner AVX2 loops.

# BLAS 2 Efficiency vs OpenBLAS

| Kernel | Matrix Size | Ours (GB/s) | % of OpenBLAS |
|--------|-------------|-------------|---------------|
| sger   | 512x512     | 201.2       | 125%          |
| ssyr2  | 512x512     | 111.5       | 140%          |
| sgemv  | 2048x2048   | 227.4       | 101%          |

- **Insight:** Our implementation of rank-updates (sger, ssyr) is exceptionally efficient because it leverages unit-stride stores across rows, creating a continuous FMA stream.

# SGEMM: 2D vs 3D Parallelism

- **TASK1 (2D):** Parallelism over Output Tiles ( $M, N$ ). Simple, low overhead.
- **TASK2 (3D):** Parallelism over Dot-Product ( $M, N, K$ ). Required for small matrices on many cores.

## Reduction Tree Implementation

To avoid global barriers, we use a task-dependency tree for  $r = 4$  K-slices:

- 1 **Level 0:** 4 independent task compute partial  $C_{m,n}$  buffers.
- 2 **Level 1:** 2 tasks sum pairs of partials.
- 3 **Level 2:** Final task adds result to matrix  $C$ .

## Intermediate Results: Thread Scaling (1024x1024)

| Threads            | GFlops (3D Task) | Efficiency          |
|--------------------|------------------|---------------------|
| 1                  | 40.9             | 100%                |
| 4                  | 110.9            | 68%                 |
| 8                  | 133.9            | 41%                 |
| <b>TASK1 (16t)</b> | <b>736.2</b>     | <b>Highest Peak</b> |

- **Analysis:** While 3D parallelism increases subtask count, the reduction overhead (memory copying) becomes a bottleneck at higher thread counts.
- **Recommendation:** Use 2D mode for  $N \geq 1024$ ; 3D mode for  $N \leq 256$ .

# Final Performance: SGEMM (2048x2048)

- **Benchmark Setup:** 16 Threads, Intel 20-core CPU.
- **OpenBLAS:** 567.4 GFlops.
- **Our Best Implementation: 837.8 GFlops.**
- **Gain: 1.48x speedup.**

## Why do we win?

Our implementation optimizes the **Macro-Kernel** for the specific L2/L3 cache geometry of the test machine and uses **Taskloop** to prevent thread starvation at the edges of the matrix.



## BLAS 3: Symmetric & Triangular Matrix Ops

- **ssyrk**:  $C = \alpha AA^T + \beta C$  (Symmetric rank-k update).
- **strmm**:  $B = \alpha op(A)B$  (Triangular matrix multiplication).
- **Optimization Strategy**:
  - Reused SGEMM micro-kernels.
  - Handled the "Half-Matrix" constraint ( $i \leq j$ ) by skipping tiles outside the boundary, saving  $\sim 50\%$  computation.
- **Result**: Correctly implemented and verified against OpenBLAS.

# Advanced 3D Optimization: Recursive Reduction Tree

- **Previous Limitation:** Special cases for  $r = 2, 4$  meant  $r = 8, 16$  reverted to serial reduction.
- **New Implementation: A Generic Binary Task Tree.**
  - Divides  $r$  slices recursively until reaching leaf tasks.
  - Summation merges happen in parallel as soon as child tasks finish.
- **Complexity:**  $O(\log r)$  critical path instead of  $O(r)$ .
- **Impact:** Enables massive parallelism in the reduction phase, vital for many-core systems where  $r$  is large.

# Closing the Loop: Automating Performance

## SGEMM Autotuner (`sgemm_tuner.py`)

- **Motivation:** Hardcoded defaults ( $MC = 120$ ,  $KC = 512$ ) are not optimal for every CPU.
- **Capability:** Searches the parameter space for  $MC$  (L2 fit),  $KC$  (L2/L3 balance), and  $NC$  (L3 residency).
- **Portability:** Allows regenerating the high-performance configuration on any machine.

## BLAS 1 Optimization

- Automatically selects unrolling and non-temporal store flags by benchmarking against a goal throughput (e.g., peak memory bandwidth).

# Testing Methodology: Ensuring 100% Reliability

| Category   | Parameters                       | Why specifically?                                                       |
|------------|----------------------------------|-------------------------------------------------------------------------|
| Edge Cases | $N \in \{0, 1, 7\}$              | Verifies boundary conditions and non-SIMD alignment logic.              |
| Strides    | $incX \in \{1, 2, 3\}$           | Tests the efficiency of unrolled scalar paths vs vectorized loads.      |
| Scalars    | $\alpha, \beta \in \{0, 1, -1\}$ | Ensures shortcuts (like skipping $C$ load if $\beta = 0$ ) are correct. |
| Hot Spots  | $N \in \{256, 512, 1024\}$       | Specifically targets L1/L2/L3 cache geometry limits.                    |

**Verification:** All 1400+ test runs compared against **Double Precision OpenBLAS** with a tolerance of  $10^{-3}$  to catch even minor FMA accumulation errors.

# Final Performance Table: Ours vs OpenBLAS

| Kernel | Size (N) | Threads | Throughput   | % of OpenBLAS |
|--------|----------|---------|--------------|---------------|
| saxpy  | $10^6$   | 16      | 43.6 GFlops  | 103%          |
| sdot   | $10^6$   | 16      | 56.4 GFlops  | <b>715%*</b>  |
| sger   | 1024     | 16      | 386.4 GB/s   | 101%          |
| ssyr2  | 512      | 16      | 111.5 GB/s   | <b>140%</b>   |
| sgemv  | 2048     | 16      | 227.4 GB/s   | 101%          |
| SGEMM  | 2048     | 16      | 837.8 GFlops | <b>148%</b>   |

\*Note: Local OpenBLAS configuration for SDOT was non-vectorized on this machine, highlighting the impact of our hand-tuned AVX2 kernel.

- **Success:** Developed a robust BLAS library covering Levels 1, 2, and 3.
- **Peak Performance:** Outperformed OpenBLAS in SGEMM and several BLAS 2 kernels.
- **Key Takeaway:** AI-driven development allows rapid exploration of unrolling factors and parallel scheduling modes (2D vs 3D) to find hardware-specific optima.